claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-ac6d47b087427e6ab.jsonl`
- SHA-256: `ce5a317380e016073652e462fa9a8ef2f37415129262151a5f744e26b0d5f034`
- Source modified: `2026-06-22T18:50:53+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T18:49:26.605Z)

```
Engine repo at C:/Users/avidu/Projects/bhi-computeval, docs/ directory. Survey existing docs to
decide WHERE this user guide should live and whether to UPDATE an existing doc or CREATE a new
one: is there a QUICKSTART / USER_GUIDE / getting-started / README that already covers
install+usage for END USERS (not developers)? Read docs/README.md (the index) + docs/CODE_MAP.md
headers + any user-facing doc. Return: a recommended filename + path for the new guide (e.g.
docs/CREATING_REELS.md or docs/USER_GUIDE.md), whether any existing doc should be updated/linked
instead, and how docs/README.md indexes docs (so the new one is added consistently). Report only
VERIFIED facts with file:line citations ? this doc must reflect SHIPPED state, never aspiration.
If something is NOT implemented, say so.
```

2. assistant (2026-06-22T18:49:29.341Z)

```
I'll survey the engine repo's docs to make this recommendation. Let me start by pulling the repo
fresh and reading the key index files.
```

3. user (2026-06-22T18:49:33.339Z)

```
Your configuration specifies to merge with the ref 'refs/heads/feat/compute-provenance'
from the remote, but no such ref was fetched.
---STATUS---
?? cloudbuild.cr.yaml
---HEAD---
ee6fba8 feat(serving): forward skip_ai_handoff to the cloud job so the picker's cloud path
yields rallies
4b6a560 feat(serving): compute-path provenance ? prove which backend ran the DETECT
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
```

4. user (2026-06-22T18:49:33.431Z)

```
No files found
```

5. assistant (2026-06-22T18:49:37.359Z)

```
The pull failed because the local checkout is on a feature branch `feat/compute-provenance`
whose remote ref isn't fetched. I'll proceed read-only on what's local (flagging staleness) and
read the index docs.
```

6. user (2026-06-22T18:49:38.695Z)

```
feat/compute-provenance
---
ABLATION_LAYER.md
```

ANALYZERS_AND_RECONCILERS.md
BACKLOG.md
CLOUD_GPU_DRYRUN_GUIDE.md
CODE_AUDIT_AND_TEST_HARDENING.md
CODE_MAP.md
COMMERCIALIZATION.md
COMPUTE_DECOUPLED_SERVING
DATA_IN_GCS.md
DEFERRED_HARDENING_PLAN.md
DESKTOP_APP_PLAN.md
DOC_STATUS.md
EVALUATION.md
EXECUTION_POLICY.md
EXPERIMENT_HARNESS.md
GOLDEN_REGRESSION_FIXTURES.md
HARDENING_LOOP_HANDOFF.md
HOSTING_PLAN.md
HOW_RALLY_DETECTION_WORKS.md
I18N_PLAN.md
MULTI_SIGNAL_FUSION_PLAN.md
NEXT_STEPS.md
NIGHTLY_ENGINE_REGRESSION.md
OWNED_MODEL_IMPLEMENTATION_PLAN.md
PACKAGING_PLAN.md
PERSONALIZATION_PLAN.md
PER_VIDEO_WORKSPACE.md
PLATFORM_ARCHITECTURE.md
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
PROJECT_DOC_TEMPLATE.md
PROMINENT_COURT_DETECTION.md
QUALITY_ITERATIONS.md
R1_MILESTONE_LAUNCH_REPORT.md
R2_EVAL_METRIC_DESIGN.md
RALLY_DETECTION_GAP_CLOSURE.md
RALLY_DETECTION_QUALITY_REPORT.md
RALLY_QUALITY_RESEARCH.md
README.md
REAL_FOOTAGE_VALIDATION_REPORT.md
ROADMAP.md
SETUP_NEW_MACHINE.md
STORAGE_SHARING_MODEL.md
TRACKNET_WSL_SETUP.md
UI_ALPHA_EXECUTION_PLAN.md
UI_PROPOSAL.md
VIDEO_LOCALITY_MODEL.md
archives
data_pipeline
mockups

7. assistant (2026-06-22T18:49:41.557Z)

The repo at `bhi-computeval` exists with a full docs directory. Let me read the index (README.md), CODE_MAP.md headers, and scan for any existing user-facing guide.

8. user (2026-06-22T18:49:42.038Z)

1   - RALLY_QUALITY_RESEARCH.md ? **? Foundation doc** for closing the local-vs-Gemini rally-detection gap: the measured gap + the code-grounded over-merge diagnosis ("shuttle moving ? rally"), an inventory of the eval/experiment harness + its gaps, the strengthened evaluation framework (segmental Edit/F1@k that make over-segmentation first-class, golden-corpus + active learning, Gemini-agreement as a *shadow* signal), an **adversarially-verified, cited external-research synthesis** (TAL/TAS boundary heads, racket-sports rally-state cues, small-data, eval rigor), a **prioritized cheap?durable roadmap with falsifiable success criteria**, and (§7) a **tracked project plan** ? work-item checklist, sequencing, the single quality **number** attributed to the effort, and a **definition of done**

(project completes when all tiers land + the number is recorded). The plan that the QUALITY_ITERATIONS.md log executes against; ties together EXPERIMENT_HARNESS / MULTI_SIGNAL_FUSION_PLAN / ANALYZERS_AND_RECONCILERS / OWNED_MODEL_IMPLEMENTATION_PLAN.

2 - RALLY_DETECTION_GAP_CLOSURE.md ? **Tracked execution doc (2026-06-18)** for closing the remaining rally-detection accuracy gap **per video, before launch**, driven by the growing golden corpus. First ground-truth clip (GX010141) reframed it: boundaries are tight (R2/R3/R4 holds) ? the gap is **detection**: (G1) recall (local & Gemini both miss ~half on hard clips, and *different* rallies ? complementary), (G2) precision (local over-fires), (G3) **rally-END over-extension on the quick-return-after-point** (shuttle still moving ? needs a *player-state* signal). Levers (owner-gated): player-kinematics + shuttle-in-hand / sustained-invisibility end-rule with reverse-timeline backtrack, `rally_gate` Gate A wiring, complementary fusion. **Peer** of `RALLY_QUALITY_RESEARCH.md` (execution tracker, not research); §7 is the source of truth.

3 - PROMINENT_COURT_DETECTION.md ? **Design + experiment (2026-06-19, default-OFF, owner-gated flip)** for the **multicourt G2 over-fire**: identify the **prominent (foreground) court** (the one covering the majority of the frame) and keep only rallies whose shuttle lives inside it ? re-defining the shuttle signal `(x,y,visible)` ? `(x,y,visible-AND-inside-prominent-court)` *before* windowing, so a background/adjacent-court shuttle never forms a rally. 2D floor polygon (MVP) ? **pseudo-3D play-volume** (refinement, so high clears aren't clipped). Born from the GX010142 rally-#1 background-court FP; reuses `fusion_features.point_in_polygon` + `FusionConfig.court_polygon`. §8 progress tracker is the source of truth.

4 - RALLY_DETECTION_QUALITY_REPORT.md ? **Technical-paper checkpoint (2026-06-17)** of rally-detection quality: the honest measured state (over-segmentation milestone tolerance-F1 0.091?0.323 at n=11, p=0.002; merge-rate 0.694?0.150; first real-footage ground-truth at-par with START at Gemini parity), the task-faithful R2 metric, a detailed Gemini comparison, next steps + expected wins, areas yet to be explored, and a publishability / path-to-submission verdict. Renders to a polished PDF; the externally-shareable "toward a paper" summary of `RALLY_QUALITY_RESEARCH.md` + `QUALITY_ITERATIONS.md`.

5 - EVALUATION.md ? How we measure and improve quality.

6 - EXPERIMENT_HARNESS.md ? Design (P1 SHIPPED) for A/B + shadow + SxS experimentation on rally-detection variants with **zero production regression** ? experiments are decoupled from deploy (new cues default OFF, promote only on measured LOVO lift, rollback = flip a flag). Composes the existing `backend/eval/` machinery (`calibration`, `regression`, `metrics`); implemented in `backend/eval/experiment.py`.

7 - QUALITY_ITERATIONS.md ? **Living log** of rally-detection quality iterations (hypothesis ? what shipped ? measured/expected effect ? lesson). The reverse-chronological record; terminal-state snapshots get archived under `archives/quality-iterations/`.

8 - ANALYZERS_AND_RECONCILERS.md ? Design (owner-gated, NOT started): a **signal-fusion framework** for rally detection ? producers emit confidence-scored **signals** at three temporal **scopes** (frame detectors / window analyzers / session analyzers), a learned **scorer** weights them, and an auditable **report-first reconciliation** pass lints the decision against the evidence. The spine: **no special-casing in the decision path** (signals?scorer, never `if/else`). Worked examples: a service-fault analyzer, a warm-up/practice span detector, and consuming a per-frame shuttle-state detector. A peer to `EXPERIMENT_HARNESS.md` and `MULTI_SIGNAL_FUSION_PLAN.md` (ADR-007 modular rally layer).

9 - PERSONALIZATION_PLAN.md ? Design (owner-adopted **hybrid**; Phase-0 landing) for the **per-user personalization feature on a generalization-first baseline**: a thin per-user *tuning* layer gated by the honest small-N statistics (a **min-N promotion floor** + a **structural-guard exemption**, never a service gate) + a **contribution flywheel** (free-tier videos pool into the owner-trained baseline; *"a tool that gets better the more you help it"*). Records the **locked owner decisions** (identity, storage, `min_n` semantics, Gate-A default, free-tier pooling). Built so far: per-user promotion gate (#141) + held-out-USER learning curve (#142).

10

11 - GOLDEN_REGRESSION_FIXTURES.md ? **Design (owner-gated, NOT started):** video-free, non-attributable regression fixtures so a clean clone runs the rally-seg suite with **no video/GPU/DB**. Two layers ? the post-detector freeze + **dual gates** (characterization + quality-floor) reusing the `backend/eval/` stack, and the **SEALED-FIXTURES** privacy filter (de-attribution-at-source, tiered public/key-gated, shuttle-only public tier). Carries a cited **IN/US/EU legal memo** (derived-data IP/privacy), a threat model, and a **§7 progress tracker + definition-of-done**. Extends [GOLDEN_DATA_SHARING.md](data_pipeline/GOLDEN_DATA_SHARING.md).

? Documentation status & governance

- DOC_STATUS.md ? **? living Documentation Status & Archival Register**: every foundational/tracked doc's lifecycle + archive-readiness, the **archival due-diligence checklist** (*honestly update ? then archive*), and the open doc-hygiene findings. Review here; update rows as docs complete.

Product / platform plans

- [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend` (what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU, in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI). Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a [runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). §7 M0?M10 tracker is the source of truth.
- [DECOUPLED_COMPUTE/](archives/past_projects/DECOUPLED_COMPUTE/README.md) ? ? **DELIVERED + ARCHIVED (2026-06-21).** Lifted the rally pipeline onto a **rented cloud-native GPU** doing bucket-driven `train` + `infer`: storage/compute seams (PRs #246?#259) + the **M3.5 native WASB runner**; **M0?M2 + M3.5 proven on a GCP L4** (M2 ? `weights/0.1.0-l4/`). Delivered on **GCP** ? DigitalOcean dropped for compute ([ADR-013](archives/decisions/DECISIONS.md)). **M3/M4 (serving endpoint + per-video billing + scale-to-zero) deferred ? the "Cloud Run + GPU serving" subproject** ([ADR-012](archives/decisions/DECISIONS.md)). Snapshot under `docs/archives/past_projects/DECOUPLED_COMPUTE/` (incl. `DEPLOY_REPORT_GCP_2026-06-20.md`).
- PACKAGING_PLAN.md ? **? Active tracked project (`IN PROGRESS`, 11 PRs merged 2026-06-21/22):** ship the engine as a **downloadable batteries-included Python app** ? `pip install` ? `indexer --app` boots a local webserver and opens the **bundled KhelSutra SPA** single-origin, **torch-free** (LIGHT tier), cross-platform, no native component (decision D10/O1). Built ON the decoupled-compute seam. **Supersedes the `DESKTOP_APP_PLAN.md` narrative.** §7 tracker = source of truth (P0a?e ?, P1a/b ?, P2-launcher/P2a/P2d/P2e ?). **? Next:** P3 GPU validation + screencast ? P2b-install ? P2c wizard (cross-repo) ? P4 (ONNX/train/annotate = north star). Memory `[[packaging-app-plan]]` / `[[next-session-packaging-app]]`.
- DESKTOP_APP_PLAN.md ? **? Superseded narrative (by `PACKAGING_PLAN.md`; D10 = self-hosted webserver, NOT a desktop app).** Original scoping doc (2026-06-18, `[design]`): package the local pipeline as a **cross-platform desktop app** for non-technical players ? plug in the camera's USB/SD card, click once, get back **only the highlight reels**, fully on-device (no upload, no original-video bloat). The centerpiece is the **de-WSL native-compute port** (run WASB natively per-OS: Linux/Windows CUDA · macOS MPS/CPU ? drop WSL2), and its **make-or-break unknown**: is CPU acceptable on a typical *no-GPU* enthusiast laptop (WASB is ~3.4× real-time on a laptop 2070S)? §7 tracker = **P0 compute-feasibility spike ? P1 native detector ? P2 app shell ? P3 packaging/installers**. Realizes the **L0 fully-local** tier of VIDEO_LOCALITY_MODEL.md; is the **`LocalDesktop` ComputeBackend** of PLATFORM_ARCHITECTURE.md §3.2; the smaller/export-clean owned model (OWNED_MODEL_IMPLEMENTATION_PLAN.md) is one answer to the compute crux.

Practical / ongoing

- SETUP_NEW_MACHINE.md ? **? The canonical "spin up a box ? ready for training & inference" runbook.** Copy-pasteable, human-followable, **verified end-to-end on DigitalOcean (2026-06-20)** ? but **compute is now GCP-only (ADR-013)**, see [`deploy/gcp/README.md`](../deploy/gcp/README.md): credentials ? provision ? bootstrap ? suite (922 passed) ? secret-free CPU inference (no GPU/keys) ? real Gemini inference ? Gen-0 training, with pointers to the GPU/native-detector + bucket paths. The `deploy/digitalocean/*` scripts are provider-agnostic and reused on GCP.
- DATA_IN_GCS.md ? **? The corpus source-of-truth map: where every piece of data lives in `gs://khelsutra-rally-corpus`** (canonical layout · current contents + drift · how `backend.worker` consumes it · the still-local-only **gap** · migration plan). Read this so a session pulls data from the bucket, not from someone's `C:`/`F:`/Drive ? the goal is to remove the local box as a blocker. Pairs with the two runbooks below.

- [`deploy/gcp/nightly_regression/README.md`](../deploy/gcp/nightly_regression/README.md) ? **? Nightly engine-regression OPERATIONS MANUAL (one-stop shop):** enable · run on demand · change any setting (email recipients/frequency/clips/config) · observe · pause/disable/remove. The full WASB-pipeline reel-count+cost+email nightly on an ephemeral GCP GPU VM (design/status: [`NIGHTLY_ENGINE_REGRESSION.md`](NIGHTLY_ENGINE_REGRESSION.md)). The lighter CPU cached-trajectory tripwire is `scripts/nightly_regression.py` (local Windows Scheduled Task, PR #202).
- TRACKNET_WSL_SETUP.md
- Evaluation harness and calibration drivers (in `backend/eval/`)

Data layer (collection · annotation · golden sets ? **no ML**)
- [`data_pipeline/`](data_pipeline/README.md) ? **the isolated data pillar**: how footage is recorded (`VIDEO_RECORDING_GUIDELINES`), golden sets are created + which videos (`GOLDEN_SET_IMPLEMENTATION` · `GOLDEN_SET_PHASE1_VIDEOS` · `GOLDEN_VIDEOS`), and data is shared/ingested (`GOLDEN_DATA_SHARING`). Deliberately **no segmentation/stitching/model** detail ? that's the ML pillar (top level + `COMPUTE_DECOUPLED_SERVING/`). The **annotation-vendor (Roora)** engagement is archived under `archives/roora-vendor/`.

Conventions & templates
- PROJECT_DOC_TEMPLATE.md ? Reusable skeleton for a **tracked project doc**: status header · decisions-locked · progress tracker (?/?/?) · definition-of-done · archived on completion. Standardizes the emergent pattern in `RALLY_QUALITY_RESEARCH.md` §7 and `GOLDEN_REGRESSION_FIXTURES.md`. Use for any substantial design/project work.

Historical documents
Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).

- CODE_AUDIT_AND_TEST_HARDENING.md ? **COMPLETED** (2026-06-14; T1?T5 + full loop + verification per #162). Living record of the June 2026 plumbing / contract / architectural audit, 626-test infrastructure, positive test plans for the 5 identified issues, multi-layer regression protocol (including ablation/experiment layer), and pre/post-change verification checklist. See `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` and `docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md` for the final archived state. (Top-level is now pointer stub per review; full content in archives/.)

- **Field-ops / customer-feedback** (recruiting flyer · field-visit playbook · PMF) ? [`archives/field-pmf/`](archives/field-pmf/) (archived 2026-06-21; PMF validation still an active *business* thread ? see `NEXT_STEPS.md`).
- **Roora annotation-vendor** engagement (labeling brief · vendor bake-off) ? [`archives/roora-vendor/`](archives/roora-vendor/) (archived 2026-06-21).
- Superseded: per-video output layout ? [`archives/PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md`](archives/PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md) (? `PER_VIDEO_WORKSPACE.md`).

See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md` for classification of small tech debt / starter projects.

9. user (2026-06-22T18:49:42.430Z)

Batteries-Included App ? Packaging & Distribution Plan

> **Status:** `IN PROGRESS` ? O1/O2 resolved (LIGHT-first; cross-platform webserver, no native component; **WASB-complete north star**); **11 PRs merged 2026-06-21/22** (P0a?e, P1a/b, P2-launcher/P2a/P2d/P2e); O3+ owner/counsel-gated · **Owner:** Avi Dullu · **Created:** 2026-06-21 · **Last updated:** 2026-06-22
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md` + `docs/NEXT_STEPS.md`; memory `[[packaging-app-plan]]` + `[[next-session-packaging-app]]`.
> **Relation to existing docs:** **supersedes** the narrative of `docs/DESKTOP_APP_PLAN.md` (per locked D10 ? self-hosted webserver, not a desktop app); **builds on** `docs/COMPUTE_DECOUPLED_SERVING/` (the profile/storage/compute seam) and is the **distribution layer** under `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.

> **Honesty note:** claims tagged `[verified]` (checked at file:line), `[researched]` (web/needs sign-off), `[design]` (proposed). Not legal advice ? licensing items need counsel.

? NEXT SESSION ? start here

You are here (2026-06-22): **v1 LIGHT is FEATURE-COMPLETE.** `pip`/`uv tool install` ? `indexer --app` boots torch-free, seeds a truly-local **`motion`** default (gate O8), and **serves the 3-step first-run wizard** single-origin (P2c, bundled), with app-home state / env-overridable ffmpeg / DNS-rebinding-guarded bind / single-instance lock / safe DB upgrades / clean install+uninstall. **P3 GPU-validated on a real L4 ($0) ? 22 rallies ? 73.7 MB reel.** Remaining = OWNER clean-box acceptance + ffmpeg-acquisition UX; then the north-star (P3-cloud/P4). Ramp up via §7 + memory `[[packaging-app-plan]]`/`[[next-session-packaging-app]]`.

Remaining work, in priority order (each its own small PR; branch off `origin/master`, explicit paths ? shared checkout):

1. ? **P3 GPU validation ? DONE (2026-06-22, $0).** Validated the packaged **wheel** end-to-end on a real L4: torch-free wheel install ? `indexer --app` ? bundled SPA ? native-WASB (separate `$WASB_PYTHON` conda env) ? **22 rallies ? 73.7 MB watermarked reel**, all via the REAL API. Proof in `gs://khelsutra-rally-corpus/highlights/p3_{packaging_highlights.mp4,run.log,app.log}`. VM deleted, $0 (~$0.17, ~14 min). The old "0 rallies" blocker was already solved by `skip_ai_handoff=true` (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`). Screencast = owner records from the preserved reel/logs.
2. ? **P2b-install ? DONE (2026-06-22).** Engine seeds a truly-local + `motion` (gate **O8**) torch-free default config on the first `indexer --app` (never overwrites); cross-platform `scripts/install.py` (`uv tool install` else pip) + per-OS launcher shortcut that pins `RALLY_APP_HOME` + `scripts/uninstall.py` driven by an install manifest (keeps user data unless `--purge`). See §7.
3. ? **P2c first-run wizard ? DONE (2026-06-22).** 3-step wizard (`SetupWizard.tsx`) from `/api/capabilities`, 6 langs, shows once/browser ? khelsutra **[#88](https://github.com/avidullu/khelsutra/pull/88)**; **re-bundled into `backend/ui/`** (`scripts/bundle_ui.sh` @ khelsutra@`7ae9cee`) so `indexer --app` SERVES it ? engine **[#312](https://github.com/Khelsutra/badminton-highlight-indexer/pull/312)**. Browser-verified e2e.
4. **? v1 LIGHT is now FEATURE-COMPLETE.** Only OWNER steps remain: a **clean Windows/macOS/Linux box ? install ? reel** acceptance run (§9 DoD) + the **ffmpeg-acquisition UX** (still a manual winget/brew/apt dep ? fold a first-run check into the wizard/`doctor.py`). Then the **north-star** (the SOLE completion goal, L10): **P3-cloud** (Cloud Run dispatch + micromamba Dockerfile ? P3a) and **P4** (ONNX export ? dissolve the dual-env; native-WASB in the default download + BYO training + in-app annotation) ? all future + counsel(O3)/owner-GPU-gated.

---

0. TL;DR

Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely by the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` / Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second, incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `$WASB_PYTHON` `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero* torch/GPU) that preserves CUJ-1..5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation.** Delivery is **cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on all three, with NO native component** (L9). **Five small engine prerequisites must land before any packaging push** (§7 P0). The single most actionable finding: **all mutable state is CWD-relative today and `--setup`/server disagree on `config.json`** ? a silent data-loss footgun that P0 must fix first.

1. Problem & goal

Goal: a non-technical user (the primary persona: an India coach/parent on Windows ?

`docs/UI_PROPOSAL.md` §2) **downloads and runs** the indexer **without sourcing the repo**. The app spins up a **local web service** for (a) inference video?rallies?reel, (b) BYO **training**, and (c) **rally annotation** if needed ? runnable **locally and against cloud services** ? and **preserves every completed CUJ** (§ CUJ map).

27

28 **Why now:** scaffolding + the decoupled-compute serving seam (M1?M8) are merged and default-OFF; the SPA + i18n + the dry-run CUJ are done. What's missing is the *distribution* story: today the only path is `git clone` ? `pip install -e .` ? out-of-band torch ? manual ffmpeg ? set up the WASB conda env ? build the SPA from a *different repo*. This plan collapses that into download-and-run.

29

30 **Read to get current:** `pyproject.toml`, `backend/main.py` (`main()` + `--server`), `backend/config/{loader,presets,models}.py`, `backend/storage/*`, `backend/compute/*`, `backend/pipeline/detectors/native_wasb_runner.py`, `deploy/cloudrun/Dockerfile`, `docs/COMPUTE_DECOUPLED_SERVING/README.md`, `khelsutra/web/` + `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.

31

32 ## 2. Decisions locked (do not relitigate)

33

34 | # | Decision | Source / date | Implication for packaging |

35 |---|----------|---------------|---------------------------|

36 | L1 | **Pure DETECT?WINDOW?STITCH core never branches on profile**; one build, profile-selected | COMPUTE_DECOUPLED_SERVING D1 | Local vs cloud is **one config flip**, not a fork ? build ON this, don't reinvent. |

37 | L2 | **Profiles** = truly-local / cloud-serving / cpu-mock via `PROFILE_PRESETS` | M1 #279 `[verified backend/config/presets.py]` | The app exposes a single "where does compute run" choice. |

38 | L3 | **Compute = GCP only** (Cloud Run GPU Jobs); DigitalOcean dropped | ADR-013, 2026-06-20 | Cloud tier targets Cloud Run + BYO-GPU-VM only. |

39 | L4 | **Edge-auth required before any non-localhost exposure** (Cloudflare Access + in-app JWT verify) | D9 / M9 `[verified backend/api/auth.py wired default-OFF main.py:294]` | Bind stays `127.0.0.1` by default; `0.0.0.0` only under an edge-auth profile. |

40 | L5 | **Delivery = self-hosted webserver the user spins up ? NOT a desktop app** | D10, 2026-06-20 (supersedes `DESKTOP_APP_PLAN.md`); **O1 confirmed 2026-06-21** | The launcher boots the server + opens the user's *own browser* at localhost ? that IS the delivery, no embedded webview. |

41 | L6 | **MIT / Apache-2.0 / BSD only** for every dependency ? code, data, AND weights | ratified 2026-06-09 | Every *new packaging* dep must clear this (see §3). |

42 | L7 | **Paid LLMs (Gemini) = inference/serving only, never a training data source**; commercial models trained on paid-LLM labels are ship-blocked | guardrail + `gate_verdict` `[verified harness.py:177 ship=False]` | The wizard's Gemini-key step must say so; training UI must surface the ship-gate. |

43 | L8 | **User-switchable profiles** ("your machine or our cloud") with explicit confirm ? cloud spend never silent | D13 / D15 | "Push to cloud" toggle needs confirm + INR estimate from `backend/billing.py`. |

44 | **L9** | **Cross-platform (Windows + macOS + Linux); NO native component** ? the same local webserver powers the UI on all three | O1, 2026-06-21 | Distribution = `uv`/pip + a **pure-Python launcher** (`webbrowser.open`). Reject native-installer-as-runtime, Electron/Tauri, and LGPL tray libs. OSes differ only in ffmpeg/torch *acquisition*. |

45 | **L10** | **North star = the FULL quality path is the SOLE completion goal** ? LIGHT is the first increment; DONE requires native WASB + BYO training + in-app annotation | O2, 2026-06-21 | §9 DoD splits "v1 shippable" from "north-star DONE"; P3/P4 (WASB/ONNX/train/annotate) are completion-gating, not optional. |

46

47 ## 3. Foundation ? research / prior art

48

49 **Distribution channels `[researched]`** ? `uv` is the only channel that can both install a pinned Python *and* route torch's `--index-url`; `pipx`/plain `pip` can't express the torch index. Frozen binaries (PyInstaller/Nuitka) with torch+CUDA are 3?5 GB, can't split CUDA libs, and **cannot host the second WASB env** ? rejected for the GPU path. Prior art (Frigate NVR ? closest analog: ffmpeg + ML + web + Docker; Ollama; ComfyUI; LM Studio) converges on **two patterns that fit us:** (1) a **container image** for the server/GPU profile, (2) a **first-run download + a friendly launcher** for the local app.

50

51 **Licensing landmines `[verified]` (the load-bearing §3 findings):**

52   - **WASB-C3 is the dominant blocker.** The WASB-SBDT *code* is MIT, but the badminton *checkpoint* provenance is **unresolved** for commercial redistribution ? `gate_verdict` hardcodes `ship=False` `[verified training/gen0/harness.py:174-177]`. **Do not conflate MIT-code with cleared-weights.** A public download must **not** auto-fetch the weights until counsel signs off (O3).
53   - **ffmpeg is GPL via the `libx264` encoder** `[verified backend/pipeline/stitcher/compiler.py:253]`. The license-clean path is **not to redistribute ffmpeg** ? resolve a system/winget install instead. `imageio-ffmpeg` also lacks `ffprobe` (which the engine needs) `[researched]`.
54   - **Anaconda ToS exposure:** the Cloud Run image uses `Miniconda3` + `conda tos accept ? pkgs/main`/`pkgs/r` `[verified deploy/cloudrun/Dockerfile:33-37]` ? the ?200-employee ToS trap. Replace with **micromamba + conda-forge** (BSD, no ToS) for the cloud image *and* any local managed WASB env; record an ADR.
55   - **New packaging deps that clear L6 `[researched]`:** `uv` (Apache/MIT), Inno Setup (permissive), `onnxruntime` (MIT), `micromamba` (BSD-3), Docker/OCI (Apache-2.0), boto3/google-cloud-* (Apache-2.0), stdlib `webbrowser`. **Forbidden:** `pystray` (LGPL-3.0). **Build-time-only:** `Nuitka` 4.x is **AGPL-3.0+** (compiled output has a runtime exception) ? usable as a tool, never bundled. `obsreport` (private git dep) stays a truly-optional extra, never bundled.
56
57   ## 4. Design / architecture
58
59   ### 4.1 One build, profile-selected (the reuse map)
60   - **Reuse:** hatchling build + `indexer = backend.main:main` console script `[verified pyproject.toml:80-82]`; `PROFILE_PRESETS` `[verified presets.py]`; `get_compute_backend(config)` returns `CloudRunCompute` only for `compute_target=='cloud_run'` else `None`=in-process `[verified backend/compute/__init__.py]`; `backend/storage/*` (local/gcs/s3/gdrive); the in-process `ThreadPoolExecutor(max_workers=1)` job worker `[verified backend/api/job_worker.py]` ? **server and worker are the same process for serving** `[verified main.py:729-736]`, so a packaged launcher just runs `indexer --server`.

10. assistant (2026-06-22T18:49:47.328Z)

The README index is detailed. I notice it does NOT list a QUICKSTART/USER_GUIDE/getting-started for end users ? the user-facing docs are all developer/runbook (`SETUP_NEW_MACHINE.md` is a "spin up a box for training/inference" runbook, not an end-user reel-creation guide). Let me verify there's no existing end-user guide and check what CUJs/CLI actually ship. Let me search for candidate names and read the CODE_MAP header plus the SPA/UI proposal.

11. user (2026-06-22T18:49:50.025Z)

=== filenames matching user-facing patterns ===
(end of filename grep)

12. user (2026-06-22T18:49:51.381Z)

Found 5 files
C:\Users\avidu\Projects\bhi-computeval\docs\archives\roora-vendor\GOLDEN_SET_VENDORING.md
C:\Users\avidu\Projects\bhi-computeval\docs\archives\MONETIZATION_AUDIT.md
C:\Users\avidu\Projects\bhi-computeval\docs\PLATFORM_ARCHITECTURE.md
C:\Users\avidu\Projects\bhi-computeval\docs\COMMERCIALIZATION.md
C:\Users\avidu\Projects\bhi-computeval\docs\I18N_PLAN.md

13. assistant (2026-06-22T18:49:54.770Z)

No filename matches any user-guide pattern. The content matches are all in strategy/vendoring docs (not end-user guides). Let me confirm by checking what those few hits actually say, and read the CODE_MAP header plus check for a top-level README and the actual shipped CLI/CUJ surface.

14. user (2026-06-22T18:49:55.727Z)

1   # CODE_MAP.md
2
3   **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-

hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; `match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`, `test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`, `test_import_guard`, `test_cvat_robustness`).

LEGEND
- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing
registry / extension point (`?`) over a new top-level file.

You're adding?	Put it in?	Notes
Backend / runtime code	`backend/<subsystem>/`	`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree.
Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)	`backend/eval/`	Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not scripts; do NOT move them out.
Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code)	**`scripts/`**	e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`.
Ops / maintenance tool (not part of the served app ? cache GC, batch admin)	`tools/`	Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested.
Training code (model fitting)	`training/`	Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`.
A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.
A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean

checkout); the no-regression gate reads it. |
| **Committed regression fixture** (video-free golden) | `eval_baselines/fixtures/<slug>/` | Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
| Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.** Per-video isolation is the tracked **`PER_VIDEO_WORKSPACE.md`** project (P0 helper shipped #264; supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) ·
`run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py`
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or `indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

---

1. PIPELINE

15. user (2026-06-22T18:49:57.768Z)

=== repo root README (top 40 lines) ===

Badminton Highlight & Rally Indexer

A local, AI-powered web application and CLI utility designed to process long-form badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality, integrity, and relevance checks), index all rallies into a relational SQLite database, and dynamically compile custom highlight reels using high-speed FFmpeg stitching.

The pipeline is built around three pluggable registries ? **validators**, **segmenters**, and **AI providers** ? plus a **sport profile** registry so the same engine can target other racket/court sports in the future. Badminton is the default profile.

> **Where this is heading:** the project is evolving from an AI-API-dependent prototype into a self-improving tool that learns rally detection *offline* from a commercially-licensed, annotated corpus. See **docs/ROADMAP.md** for the four-phase plan and **docs/archives/decisions/DECISIONS.md** (historical ADRs) for the rationale.

? Key Features

*   **Pluggable Video Validation Framework**: Easily extendable base-class architecture that automatically runs checks on:
    *   *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and framerate >= 30 FPS.
    *   *PTS Timeline Continuity*: Scans keyframe timestamps for timeline gaps, jumps, or editing splices to detect tampering or splices.
    *   *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
    *   *Blurriness Check*: Calculates average frame Laplacian variance to reject out-of-focus footage.
    *   *Relevance (Is it Badminton?)*: Uses high-performance HSV color court matching to identify court layouts.
*   **Six Pluggable Rally Segmenters** (`backend/pipeline/segmenters/` ? `motion`, `gemini`,

`yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`). The three core ones:
    *   `yolo_hybrid` ? two-stage pipeline. Stage 1 runs **torchvision person detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-licensed) to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive components ? see docs/archives/MONETIZATION_AUDIT.md and ADR-005.)*
    *   `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini directly. Highest precision per call, highest cost.
    *   `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key required. Useful as a baseline or for offline/air-gapped use.
*   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without touching the segmenters.
*   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and registering it.
*   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments (start/end/duration/server/receiver/metadata) and per-segment events are persisted in `output/sports_indexer.db`. Re-running on the same file is a cache hit.
*   **Query System**: Filter indexed play segments by `server`, `receiver`, `duration_min`, and `event_type` (the schema supports rich event types like smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated `shot_count`).
*   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip stitching.
*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`, and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel (**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with an off switch).
*   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics, validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an arbitrary segment via stream-copy ffmpeg for fast iteration.

?? Setup & Diagnostics

0. Batteries-included install (LIGHT tier ? recommended for end users)

For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock), writes a launcher script (in your app-data folder), and seeds a sensible default config.

```
=== console_scripts / CLI entry ===
1:# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
17:name = "badminton-highlight-indexer"
84:[project.scripts]
85:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86:indexer = "backend.main:main"
```

16. assistant (2026-06-22T18:50:02.920Z)

This is the key finding: the **root `README.md` already contains substantial end-user-facing install + usage content** ("Setup & Diagnostics", "Batteries-included install (LIGHT tier ? recommended for end users)"). Let me read more of the root README to understand exactly how much end-user usage it covers, and check the UI_PROPOSAL/UI_ALPHA docs since the actual reel-creation flow is the SPA, not the CLI.

17. user (2026-06-22T18:50:05.021Z)

=== root README line count ===

```
449 README.md

=== README section headers ===
1:# Badminton Highlight & Rally Indexer
11:## ? Key Features
33:## ?? Setup & Diagnostics
35:### 0. Batteries-included install (LIGHT tier ? recommended for end users)
42:# Preferred ? uv installs `indexer` into an isolated tool env:
44:# If `indexer` isn't found afterwards, add uv's tool-bin to PATH once: uv tool update-shell
46:# Or the cross-platform installer script (uv-if-present, else pip) ? also writes a launcher
47:# script (with the resolved `indexer` path baked in) + an uninstall manifest:
67:### 1. External System Requirements
76:### 2. Install the package
82:# Runtime deps only:
84:# Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
102:### 3. Diagnostics Checker (`scripts/doctor.py`)
107:# or, via the CLI:
120:## ? Running the Application
122:### 1. Launching the Local Web App Dashboard
130:### 2. Running in CLI Processing Mode
144:### 3. Rich CLI Debugging (`--debug-clip`)
151:### 4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest
watermark preview
164:### 5. End-to-End "Process + Stitch" Script
167:### 6. Cache Hygiene & Disk Cleanup
182:## ?? Threshold Tuning (`config.json`)
254:## ? Frequently Asked Questions (FAQ)
256:### Q1: The server complains that `ffprobe` is not found.
259:### Q2: Why does my video fail the Relevance check?
262:### Q3: How do I add a new custom validator to the pipeline?
268:### Q4: Why does the CLI or server output freeze or take a long time on large videos?
275:### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
278:### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB
video file?
285:### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
309:### Q8: How does the `yolo_hybrid` pipeline actually save money?
319:### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
332:### Q10: Can I trust the per-segment metadata from the `motion` segmenter?
335:### Q11: I re-ran a video and my previous results changed/disappeared. Why?
338:### Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?
346:### Q13: A validation check failed and indexing was halted ? what do I do?
349:### Q14: The report says fps was a "fallback" (not probed) ? what does that mean?
360:## ? Annotation & Evaluation
366:# ...fill in start,end rows, then:
376:## ? Running the Test Suite
382:# One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
383:# runs the whole suite, reports a clear PASS/FAIL, and returns pytest's exit code:
387:# ...or call pytest directly:
412:## ? Project Structure (high level)
```

18. user (2026-06-22T18:50:05.783Z)

```
1       # UI Proposal ? khelsutra.guru web app (alpha ? beta)
2
3       **Status:** PROPOSAL ? awaiting owner ratification (no implementation starts without
explicit approval, per CLAUDE.md).
4       **Date:** 2026-06-10 · **Inputs:** PMF_MARKET_RESEARCH, COMMERCIALIZATION,
PLATFORM_ARCHITECTURE, DEFERRED_HARDENING_PLAN (#16), ROADMAP, VIDEO_RECORDING_GUIDELINES,
reporting/spec.yaml, current API + static UI.
5
6       > **? Partly shipped + in implementation (2026-06-21):** the alpha SPA now lives in
**`khelsutra/web/`**
7       > (the per-rally console shipped) and the marketing site in `khelsutra/site/`. The
**non-tech-savvy UX** half of
```

> this proposal is tracked in **`khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md`** (friendly + Hindi/Kannada). Treat
> the screens below as design intent; the live state is in that repo. *(Resolves `DOC_STATUS.md` finding #5 for
> this doc; the static-UI references are historical ? that UI is retired.)*
Companion: `docs/HOSTING_PLAN.md` (domain, topology, costs).

1. Strategic frame ? why a UI now, and what it secretly is

The UI is not just product surface; it is the **corpus-growth engine**. The single highest-leverage
item in NEXT_STEPS is growing the golden corpus, and the v1 scope already commits to a
human correction loop. Every tester who accepts/rejects/nudges a rally boundary in the review
screen produces exactly the labels the Gen-0/M0.x track is starved for (subject to the per-user
training opt-in guardrail). 10?25 alpha testers ? a steady stream of labeled, real-world,
multi-court-biased footage ? the distribution where the owned model's 0.66 ship target lives.

Serving economics per the 2026-06-10 quality table: alpha serves the **Gemini path**
(0.93 on clean footage, ~$0.05/run) ? no GPU required in the request path. The WASB/owned-model
path stays on the owner's box and is *not* exposed in the alpha UI (segmenter choice hidden).

The CLI is not replaced. The web app talks to the same FastAPI surface the CLI drives; the CLI
remains the power-user/ops tool.

2. Personas (from PMF research)

- **Coach / academy (primary, B2B, India):** batches of match recordings, wants rally review,
  per-player reels for students/parents, per-rally stats. Already records on static cameras.
- **Enthusiast (B2C funnel):** records own matches, wants a highlight reel + basic stats, shares
  with friends. Tripod discipline is the known adoption risk ? the UI must teach recording
  guidelines *before* upload, not fail videos after.

3. ALPHA ? "quick & dirty" (goal: 10?25 invited testers, end-to-end, zero hand-holding)

Stack: React + Vite + Tailwind SPA, deployed per HOSTING_PLAN (static hosting, $0).
API = existing FastAPI on the GPU box via tunnel at `api.khelsutra.guru`.

19. assistant (2026-06-22T18:50:11.229Z)

Now I have a complete picture. The root `README.md` (449 lines) is the de-facto end-user + developer install/usage guide, but it's a single mixed-audience doc heavy on CLI, config, FAQ, and test-suite ? not a clean "create a highlight reel" walkthrough for a non-technical user via the SPA. Let me read the README's "Running the Application" and "Batteries-included install" sections in full to nail the exact shipped state, and confirm what the SPA-based reel flow looks like (which lives in the khelsutra repo, not here).

20. user (2026-06-22T18:50:11.910Z)

0. Batteries-included install (LIGHT tier ? recommended for end users)

For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform
installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock),

writes a launcher script (in your app-data folder), and seeds a sensible default config.

```bash
# Preferred ? uv installs `indexer` into an isolated tool env:
uv tool install badminton-highlight-indexer        # (once published) or:  uv tool install .
# If `indexer` isn't found afterwards, add uv's tool-bin to PATH once: uv tool update-shell

# Or the cross-platform installer script (uv-if-present, else pip) ? also writes a launcher
# script (with the resolved `indexer` path baked in) + an uninstall manifest:
python scripts/install.py                            # add --method pip to force pip, --dry-run to preview
```

Then launch the app (the installer's shortcut, or directly):

```bash
indexer --app        # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
```

* **First run seeds a truly-local default** ? detector `motion` (offline, **no API key, no GPU,
  no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels) lives
  in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` /
  `~/.local/share/Khelsutra`) when launched via the installer's shortcut.
* **FFmpeg is still required** (see step 1) ? it is intentionally not redistributed (GPL `libx264`).
* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your config/reels
  by default; add `--purge` to delete them too.

The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see `docs/PACKAGING_PLAN.md`.

1. External System Requirements
The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes, and stitch clips.

*   **On Windows (PowerShell as Admin)**:
    ```powershell
    winget install Gyan.FFmpeg
    ```
    Restart your terminal afterwards to register the path changes.

2. Install the package

The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling
build backend). Install it editable:

```bash
# Runtime deps only:
pip install -e .
# Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
pip install -e .[dev]
```

This registers the `indexer` console entry point (so `indexer --server`,
`indexer --help`, etc. work) and puts the `backend` and `training` packages on
your path.

> **PyTorch is installed separately.** `torch`/`torchvision` are intentionally
> *not* in `pyproject.toml`: their CPU/CUDA wheels need an out-of-band wheel index
> that PEP 621 cannot express. Install them yourself, matching your hardware:
> ```bash
> # NVIDIA GPU (CUDA 12.4):
> pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url https://download.pytorch.org/whl/cu124
> # CPU only:
> pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
> ```

3. Diagnostics Checker (`scripts/doctor.py`)
Run the diagnostics checker to verify your system dependencies, initialize a
default config, and (optionally) bootstrap the Python packages:
```bash
python scripts/doctor.py
# or, via the CLI:
indexer --setup
```
This script will:
*   Validate `ffmpeg` and `ffprobe` are present in your PATH.
*   Initialize a default `config.json` (if one does not exist) with thresholds for blur, court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
*   Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
*   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`, `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`, `supervision`.

> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ? nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`: `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or `retinanet_resnet50_fpn`.)

? Running the Application

1. Launching the Local Web App Dashboard
To start the FastAPI server and open the web dashboard:
```bash
python -m backend.main --server --port 8000
```
Open your browser and navigate to **http://localhost:8000**.
Simply input your local raw video file path in the sidebar to process, validate, index, and compile highlights instantly.

2. Running in CLI Processing Mode
To process and index a video completely via terminal commands:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4"
```
This outputs a validation summary and indexes all rallies directly into the local SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently produces 0 segments and the observability report flags `silent_provider_noop`). Set it to `yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.

To explicitly pick a segmenter for one run:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
```

```

### 3. Rich CLI Debugging (`--debug-clip`)
To diagnose exactly why a video segment was registered as active/dead play, or inspect detailed focus/continuity metrics at a particular time in seconds:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
```
Outputs granular validation check logs and motion ratios within a +/- 3 second frame window around the timestamp.

4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest watermark preview
Extract a slice of a recording to disk (re-encoded; and, with the default-on branding watermark, stamped **"Generated by Khelsutra.guru"** in the corner):
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" \
    --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
```
Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute path. This is the fastest way to *see* the watermark on a real clip ? set `stitching.watermark.enabled: false` in `config.json` to turn it off.

> To preview the exact overlay on any file without the pipeline:
> ```bash
> ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu Sans':fontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-24:box=1:boxcolor=black@0.4:boxborderw=10" -c:a copy out.mp4
> ```

5. End-to-End "Process + Stitch" Script
`scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a hardcoded video path, reuses cached indexed segments if available (offering to re-index from scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`, `min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching settings against a fixed large recording without re-running expensive segmentation.

6. Cache Hygiene & Disk Cleanup
The trajectory detectors (`wasb_hybrid` / `tracknet_hybrid`) stage a copy of each video into WSL and decode every frame to PNG ? easily **tens of GBs per 4K video**. To keep disk usage bounded:

*   **Automatic self-clean:** after a *successful* run the frame cache + staged copy are deleted (`indexing.{wasb,tracknet}.keep_frames`, default `false`). On failure they're kept so a re-run resumes from the checkpointed detector cache.
*   **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with `--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and `output/`:
    ```bash
    python -m tools.cleanup_caches                  # report what's reclaimable (safe; deletes nothing)
    python -m tools.cleanup_caches --apply          # delete the regenerable caches
    python -m tools.cleanup_caches --keep-video GX010125   # protect one video's cache (id or filename stem)
    python -m tools.cleanup_caches --older-than 7   # only purge entries older than 7 days
    ```
    It treats frame caches, staged/proxy videos, and handoff/temp dirs as **regenerable** (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels. Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line nudge (used by the optional SessionStart hook).

21. assistant (2026-06-22T18:50:15.349Z)
    ```

I now have everything needed for a verified recommendation. Let me do two final confirmations: that `indexer --app` and `--server` flags actually exist in code (the README claims them), and how DOC_STATUS treats the README (whether it's a tracked doc).

22. user (2026-06-22T18:50:18.402Z)

```
=== --app / --server / --setup flags in backend/main.py ===
1136:    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and setup utility")
1138:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
1139:    parser.add_argument("--app", action="store_true",

=== README mentioned in DOC_STATUS or README index? ===
docs/DOC_STATUS.md:21:3. **Update ALL inbound references** ? `docs/README.md` index, `CLAUDE.md`, `docs/NEXT_STEPS.md`, and any cross-doc links ? the new path + status.
docs/DOC_STATUS.md:26:> **Worked example (the template):** **DECOUPLED_COMPUTE ? #270** ? verified M0?M2+M3.5 delivered on GCP, flipped ??? DELIVERED with a completion note (incl. carried-forward WASB-C3/DPA/`md5`), reindexed `docs/README` + `past_projects/README`, kept the ADR-010?014 lineage, then `git mv` to `docs/archives/past_projects/DECOUPLED_COMPUTE/`.
docs/DOC_STATUS.md:44:| **Data layer** ? `docs/data_pipeline/` (`GOLDEN_SET_IMPLEMENTATION` · `GOLDEN_SET_PHASE1_VIDEOS` · `GOLDEN_VIDEOS` · `VIDEO_RECORDING_GUIDELINES` · `GOLDEN_DATA_SHARING`) | LIVING (data, **no-ML**) | n/a | Relocated 2026-06-21 to the isolated data pillar; tracked via `data_pipeline/README.md`. Roora *vendor* docs archived ? `archives/roora-vendor/`. |
docs/DOC_STATUS.md:73:`PROJECT_DOC_TEMPLATE.md` · `NEXT_STEPS.md` (? refreshed 2026-06-21) · `BACKLOG.md` · `EVALUATION.md` · `ABLATION_LAYER.md` · `SETUP_NEW_MACHINE.md` · `TRACKNET_WSL_SETUP.md` · `README.md`. *(2026-06-21 relocation: the data-layer guides moved ? `data_pipeline/`; the field-ops + Roora docs ? `archives/{field-pmf,roora-vendor}/`.)*
docs/README.md:17:- [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend` (what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU, in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI). Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a [runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). §7 M0?M10 tracker is the source of truth.
docs/README.md:18:- [DECOUPLED_COMPUTE/](archives/past_projects/DECOUPLED_COMPUTE/README.md) ? ? **DELIVERED + ARCHIVED (2026-06-21).** Lifted the rally pipeline onto a **rented cloud-native GPU** doing bucket-driven `train` + `infer`: storage/compute seams (PRs #246?#259) + the **M3.5 native WASB runner**; **M0?M2 + M3.5 proven on a GCP L4** (M2 ? `weights/0.1.0-l4/`). Delivered on **GCP** ? DigitalOcean dropped for compute ([ADR-013](archives/decisions/DECISIONS.md)). **M3/M4 (serving endpoint + per-video billing + scale-to-zero) deferred ? the "Cloud Run + GPU serving" subproject** ([ADR-012](archives/decisions/DECISIONS.md)). Snapshot under `docs/archives/past_projects/DECOUPLED_COMPUTE/` (incl. `DEPLOY_REPORT_GCP_2026-06-20.md`).
docs/README.md:23:- [SETUP_NEW_MACHINE.md](SETUP_NEW_MACHINE.md) ? **? The canonical "spin up a box ? ready for training & inference" runbook.** Copy-pasteable, human-followable, **verified end-to-end on DigitalOcean (2026-06-20)** ? but **compute is now GCP-only (ADR-013)**, see [`deploy/gcp/README.md`](../deploy/gcp/README.md): credentials ? provision ? bootstrap ? suite (922 passed) ? secret-free CPU inference (no GPU/keys) ? real Gemini inference ? Gen-0 training, with pointers to the GPU/native-detector + bucket paths. The `deploy/digitalocean/*` scripts are provider-agnostic and reused on GCP.
docs/README.md:25:-
[`deploy/gcp/nightly_regression/README.md`](../deploy/gcp/nightly_regression/README.md) ? **? Nightly engine-regression OPERATIONS MANUAL (one-stop shop):** enable · run on demand · change any setting (email recipients/frequency/clips/config) · observe · pause/disable/remove. The full WASB-pipeline reel-count+cost+email nightly on an ephemeral GCP GPU VM (design/status: [`NIGHTLY_ENGINE_REGRESSION.md`](NIGHTLY_ENGINE_REGRESSION.md)). The lighter CPU cached-trajectory tripwire is `scripts/nightly_regression.py` (local Windows Scheduled Task, PR #202).
docs/README.md:30:- [`data_pipeline/`](data_pipeline/README.md) ? **the isolated data pillar**: how footage is recorded (`VIDEO_RECORDING_GUIDELINES`), golden sets are created + which videos
```

(`GOLDEN_SET_IMPLEMENTATION` · `GOLDEN_SET_PHASE1_VIDEOS` · `GOLDEN_VIDEOS`), and data is shared/ingested (`GOLDEN_DATA_SHARING`). Deliberately **no segmentation/stitching/model** detail ? that's the ML pillar (top level + `COMPUTE_DECOUPLED_SERVING/`). The **annotation-vendor (Roora)** engagement is archived under `archives/roora-vendor/`.
docs/README.md:44:See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md` for classification of small tech debt / starter projects.

23. assistant (2026-06-22T18:50:53.725Z)

I have all verified facts. Note `docs/README.md` is a flat doc index (no top-level title/intro heading ? it begins mid-list at line 1 with `RALLY_QUALITY_RESEARCH.md`), and the root README is governance-listed as a LIVING doc in `DOC_STATUS.md:73`. Here is my recommendation.

Recommendation

Is there an existing end-user install+usage guide?
Partially ? in the root `README.md`, not in `docs/`. There is **no** `QUICKSTART` / `USER_GUIDE` / `getting-started` doc anywhere under `docs/` (verified: filename grep for those patterns returns nothing; the only content hits for "quickstart/user guide" are in strategy docs `PLATFORM_ARCHITECTURE.md`, `COMMERCIALIZATION.md`, `I18N_PLAN.md`, and archived vendoring docs ? none are end-user reel guides).

What *does* cover install+usage for end users today is the repo-root **`README.md`** (449 lines, `README.md:1-449`). It already has:
- An explicit **"Batteries-included install (LIGHT tier ? recommended for end users)"** section (`README.md:35-65`) covering `uv tool install` / `scripts/install.py` ? `indexer --app` (CLI flag verified to exist at `backend/main.py:1139`), app-data state dirs, ffmpeg caveat, and clean uninstall.
- **"Running the Application"** (`README.md:120-181`) ? launching the web dashboard (`indexer --server`, flag verified `backend/main.py:1138`), CLI processing mode, and reel/trim/watermark usage.
- A 14-question **FAQ** (`README.md:254-359`) and config-tuning reference (`README.md:182-253`).

But it is a mixed-audience doc ? heavily weighted toward developers/power-users (editable `pip install -e .`, PyTorch wheel indices, `--debug-clip`, `config.json` threshold tuning, the test suite, project structure). It is **not** a clean, task-first "non-technical user creates a highlight reel" walkthrough, and the actual friendly reel-creation flow is the **SPA**, which lives in the *sibling* `khelsutra` repo, not this engine repo (`UI_PROPOSAL.md:6-10` confirms the alpha SPA moved to `khelsutra/web/` and the non-tech UX is tracked in `khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md`). So this engine repo has no dedicated end-user guide for the reel-creation user journey.

Recommended placement for the new guide
Create a new doc: `C:/Users/avidu/Projects/bhi-computeval/docs/CREATING_REELS.md` (a task-oriented "install ? process a video ? pick rallies ? get a watermarked reel" guide for end users via `indexer --app`).

- Prefer `CREATING_REELS.md` over `USER_GUIDE.md`: it is action/outcome-named (matches this repo's doc-naming style ? `PACKAGING_PLAN`, `SETUP_NEW_MACHINE`, `CLOUD_GPU_DRYRUN_GUIDE`) and avoids colliding conceptually with the root README, which is already the catch-all "user guide."
- Do **not** name it `QUICKSTART.md` if it will be more than a few steps; reserve that only if you want a deliberately short page.

Should an existing doc be updated/linked instead?
- **Do not duplicate** the root `README.md` install section ? **link to it.** The new guide should reference the root README's "Batteries-included install" (`README.md:35-65`) for install mechanics and focus on the *usage journey* (open the app, point at a video, review/pick rallies, download the reel), not re-derive install.
- **Scope check (verified, not aspirational):** `indexer --app` ships and boots the bundled SPA single-origin (`backend/main.py:1139`; `PACKAGING_PLAN.md:12` "v1 LIGHT is FEATURE-COMPLETE"). The **native-GPU (WASB) and cloud serving tiers are owner-only / not in the default download** (`README.md:65`, `PACKAGING_PLAN.md:18,23`) ? the guide must say the default LIGHT tier is

Gemini/`motion`/cpu-mock only and NOT promise on-device GPU quality or one-click cloud. **ffmpeg is a manual prerequisite** (`README.md:61,67-74`) ? not auto-installed; call this out as a required step.
- Cross-repo note: the friendly multilingual reel UX itself is implemented in the sibling `khelsutra` repo (`UI_PROPOSAL.md:6-10`). If the guide documents on-screen SPA steps, flag that the UI is bundled from there so screenshots/wording stay in sync.

How `docs/README.md` indexes docs (so the new one is added consistently)

`docs/README.md` is a **flat, single-level bulleted index grouped under `##` section headings** (verified `README.md:1-44`). Each entry is one bullet of the exact form:

```
- [FILENAME.md](FILENAME.md) ? **<bold status/tag>** one-paragraph description (what it is, status, source-of-truth pointer).
```

Sections present: *(top, untitled list)* ? `## ? Documentation status & governance` ? `## Product / platform plans` ? `## Practical / ongoing` ? `## Data layer ?` ? `## Conventions & templates` ? `## Historical documents`. There is **no H1/intro** ? the file opens directly at line 1 with the `RALLY_QUALITY_RESEARCH.md` bullet.

Add the new guide under `## Practical / ongoing` (alongside `SETUP_NEW_MACHINE.md` at `README.md:23`, which is the dev/ops runbook ? the new doc is its end-user counterpart), as e.g.:
`- [CREATING_REELS.md](CREATING_REELS.md) ? **? End-user guide:** install the LIGHT-tier app (? root README §0) ? indexer --app ? process a video ? pick rallies ? download a watermarked reel. LIGHT tier only (Gemini/motion/cpu-mock; no on-device GPU/cloud). Pairs with PACKAGING_PLAN.md.`

Also: governance convention per `CODE_MAP.md:33` ("A doc ? Index it in `docs/README.md`") and `DOC_STATUS.md:21` ? when you add it, list it in `docs/README.md` and add a row/mention to `docs/DOC_STATUS.md` (the README itself is already tracked there as LIVING at `DOC_STATUS.md:73`).

Verification caveat

`git pull --ff-only` **failed** ? the local checkout is on branch `feat/compute-provenance` whose remote ref wasn't fetched (HEAD `ee6fba8`), so these facts reflect the **local checkout, which may lag/lead `origin/master`**. All `file:line` citations above were read from the working tree; re-verify the README line ranges against `origin/master` before writing, since the root README is a LIVING doc that moves between sessions.